# ■ AidFlow Pro

## AI-Powered Humanitarian Aid Distribution Platform

**Competition:** Gemma 4 Good Hackathon — Kaggle × Google DeepMind

**Track:** Global Resilience | **Prize Pool:** $200,000 | **Deadline:** May 18, 2026

**AI Model:** Gemma 4 E5B IT (local) | **Version:** 1.0 | **Date:** May 2026

---

*Bringing AI-powered triage and offline-first intelligence to humanitarian crisis zones — so every family gets the right aid, at the right time, even without internet.*

## Table of Contents

## 1. EXECUTIVE SUMMARY

AidFlow Pro is an offline-first web application that empowers humanitarian aid organizations to distribute aid intelligently, efficiently, and equitably to impacted families — even in areas with no internet connectivity. The platform's intelligence core is powered by **Gemma 4 E5B IT**, Google's open-weight, locally-runnable AI model, which provides real-time prioritization, contextual recommendations, and natural-language querying over the aid database.

The application integrates **Bitchat** (a Bluetooth mesh messaging protocol) for offline field communications, an embedded **Starlink equipment coverage map** for connectivity planning, and a comprehensive document knowledge base that allows organizations to upload PDF guides which Gemma 4 can learn from and reference. AidFlow Pro addresses the Gemma 4 Good Hackathon's *Global Resilience* track and demonstrates how frontier open-source AI can operate without cloud dependency in the world's most challenging environments.

| | |
|---|---|
| Primary AI Model | Gemma 4 E5B IT (locally hosted via Ollama) |
| Connectivity Modes | Online (API + internet) & Fully Offline (local DB + local AI) |
| Languages Supported | English, Arabic, French, Spanish |
| Core User Types | Organization Admins, Field Workers, Data Managers |
| Hackathon Track | Global Resilience ($200,000 Prize Pool, Deadline May 18, 2026) |
| Submission Requirements Met | Working demo, public code repo, technical write-up, video |

## 2. PROBLEM STATEMENT

Humanitarian aid distribution in crisis zones faces three critical failure points that AidFlow Pro solves:

- **Inequitable Distribution:** Without data-driven prioritization, the most vulnerable families — malnourished children, pregnant mothers, the elderly — often receive aid in the same cycle as less critical households, wasting scarce resources.
- **Connectivity Dependency:** Most existing aid management software (SCOPE, LogistiX, KoBoToolbox) requires persistent internet access, making them unusable in disaster zones where infrastructure is destroyed.
- **Communication Blackouts:** Field teams cannot coordinate with affected families when cellular networks go down. This leads to mis-deliveries, duplicated distribution, and complete communication failure in the field.
- **Knowledge Inaccessibility:** Organizational guides, medical protocols, and disaster response manuals exist as PDFs stored in headquarters — unavailable to field workers when they need them most.
- **No Dynamic Updates:** Once aid is delivered, family status changes (new medical need, family member death, displacement) are not reflected in priority scores, causing outdated and unfair future distributions.

## 3. PROJECT OVERVIEW & VISION

**Project Name: AidFlow Pro**

**One-Line Vision:** An AI-powered, offline-capable humanitarian aid distribution platform that ensures the right aid reaches the right family at the right time — always.

AidFlow Pro is designed as a **Progressive Web Application (PWA)** — it installs like a native app on any device, caches all critical data and the AI model locally, and functions fully offline. When internet becomes available, it synchronizes data and enables Gemma 4 to access the web for enriched intelligence.

### User Roles

| Role | Responsibilities |
|---|---|
| Organization Admin | System configuration, user management, bulk data import, report generation |
| Field Supervisor | Oversee distribution sessions, approve priority overrides, review AI recommendations |
| Field Worker | Execute distributions, update family records, use Bitchat to communicate |
| Data Manager | Upload PDFs (guides, protocols), manage knowledge base, add training content |
| Read-Only Viewer | View dashboards and distribution history (auditors, donors) |

## 4. CORE FEATURES & MODULES

### 4.1 AI-Powered Aid Prioritization Engine (Gemma 4)

The heart of AidFlow Pro. Gemma 4 E5B IT runs locally via Ollama and analyzes each family's profile in the backend database to compute a real-time Priority Score (0–100).

- Reads family data: member count, ages, health conditions, last aid received, displacement status, income level
- Computes composite priority using AI reasoning + configurable weight rules
- Explains every prioritization decision in plain language (e.g., 'Family #0042 is Priority 1: 3 children under 5, no food aid in 14 days')
- Natural language queries: ask 'Which families need medical supplies in Sector B?' in any supported language
- Operates fully offline using locally stored database snapshot + Gemma 4 model weights
- When online: enriches decisions using web search (e.g., local disease outbreaks, weather alerts)

### 4.2 Distribution Execution & Record Update Module

Field workers execute AI-recommended distributions and update family records in real-time, triggering priority score recalculation.

- Checklist-based distribution interface optimized for tablets and low-end phones
- Barcode / QR code scanning to identify families (works offline)
- Real-time update form: mark items delivered, add notes, flag special needs
- After update: Gemma 4 recalculates priority score with the new status

- Conflict detection: alerts if a family received aid recently (duplicate prevention)
- Full offline write support — updates queue locally and sync when online

## 4.3 Knowledge Base & PDF Document Upload

Organizations upload PDF guides (starvation protocols, medical triage, aid item usage guides) which are processed into a local vector store. Gemma 4 uses RAG (Retrieval-Augmented Generation) to reference these documents when making recommendations.

- Drag-and-drop PDF upload interface with category tagging (Medical, Food, Shelter, etc.)
- PDFs are chunked and embedded using a local embedding model (no cloud required)
- Vector store persisted in IndexedDB (browser) or local SQLite (server mode)
- Gemma 4 cites the source document and page number when referencing guide content
- Supports WHO guidelines, ICRC protocols, NGO-specific manuals
- Document versioning — new uploads supersede old ones by category

## 4.4 Emotional Support Content Library for Children

A dedicated media library for uploading and displaying age-appropriate emotional support content for children in crisis zones.

- Upload images, short videos, interactive PDFs (coloring books, games, stories)
- Categorized by age group (0–5, 6–10, 11–15) and language
- Offline-accessible: content cached locally on field devices
- Field workers can share content with families via QR code or Bitchat
- Gemma 4 can suggest appropriate content based on child age and trauma indicators

## 4.5 Starlink Equipment Map Module

An embedded interactive map showing nearby Starlink equipment sellers/providers and estimated coverage areas, helping organizations plan connectivity before field operations.

- OpenStreetMap + Leaflet.js base map cached via Service Worker for full offline use
- Starlink provider data sourced from a curated community dataset (preloaded, no API required)
- Coverage area polygons rendered as color-coded overlays (Strong/Moderate/Weak signal)
- User can add custom provider pins with notes (offline)
- Filters: by country, region, provider type, availability status
- Links to provider contact info and nearest service point routing

## 4.6 Bitchat Offline Communication Integration

AidFlow Pro embeds a web-based Bitchat client, enabling Bluetooth mesh messaging between field teams and families when internet is unavailable. Uses Bitchat's open protocol (permissionlesstech/bitchat).

- In-app chat interface with channels per operational zone
- Bluetooth LE mesh: messages relay up to 7 hops through nearby devices

- When internet available: falls back to Nostr protocol (290+ global relays)
- End-to-end encryption via Noise Protocol (offline) and NIP-17 gift-wrapping (online)
- Messages are ephemeral by default — no persistent server logs
- Integration note: Web Bluetooth API required (Chrome/Edge on desktop, Android Chrome)

## 4.7 Aid Usage Guides Module

Separate from the AI knowledge base — a structured library of how-to guides for specific aid items (e.g., how to use a water purification kit, how to assemble emergency shelter).

- Organization staff upload guides as PDF, video link, or rich text
- Searchable by aid item name, category, or language
- Field workers can pull up the guide for any item being distributed
- Gemma 4 can summarize or answer questions about any guide in plain language
- Available fully offline after initial sync

## 4.8 Multilingual Interface & AI Responses

The full application UI and all Gemma 4 AI outputs are available in 4 languages, with runtime switching.

- Supported languages: English (EN), Arabic (AR, RTL layout), French (FR), Spanish (ES)
- i18n implemented with react-i18next — all UI strings externalized
- Gemma 4 instructed via system prompt to respond in the user's active language
- RTL (Right-to-Left) layout toggle for Arabic UI
- Language preference stored locally per device

## 5. TECHNICAL ARCHITECTURE

AidFlow Pro follows a **Progressive Web App (PWA) + Local-First Architecture**. The frontend is a React SPA that communicates with a lightweight local Node.js/FastAPI backend running on the organization's server or laptop. Gemma 4 runs as a local Ollama instance. All critical data is cached in IndexedDB via a Service Worker, enabling complete offline operation.

### Architecture Layers

| Layer | Components | Offline? |
|---|---|---|
| Frontend (PWA) | React 18 + Vite, TailwindCSS, react-i18next, Leaflet.js, Web Bluetooth API | ■ Full |
| Service Worker | Workbox — precaches app shell, map tiles, model configs, static assets | ■ Full |
| Local Storage | IndexedDB (app data, chat queue, upload queue), LocalStorage (prefs) | ■ Full |
| API Layer | FastAPI (Python) or Express.js — REST endpoints, WebSocket for live updates | ■ Full |
| AI Engine | Ollama serving Gemma 4 E5B IT on localhost:11434 — OpenAI-compatible API | ■ Full |
| RAG Pipeline | LangChain + ChromaDB (local vector store) — PDF chunking and embedding | ■ Full |
| Assumed Backend DB | PostgreSQL (assumed existing) — read via REST API, sync to local SQLite | ■ Cached |
| Bitchat Layer | Web Bluetooth API bridge to Bitchat Bluetooth mesh protocol | ■ Full |
| Online Enhancement | Web search via Gemma 4 function calls (when internet available) | ■ Online only |
| Map Tiles | OpenStreetMap tiles cached offline via Service Worker (Leaflet.js) | ■ Full |

### Data Flow Diagram (Described)

The following describes the primary data flow for a distribution session:

1. Field Worker opens AidFlow Pro PWA on tablet/laptop (fully offline capable)
2. App loads cached family data from IndexedDB (last synced from backend DB via REST API)
3. User requests AI prioritization → Frontend sends request to local FastAPI server
4. FastAPI calls Ollama (localhost:11434/v1/chat/completions) with family data context + RAG-retrieved guide excerpts
5. Gemma 4 E5B IT processes the request locally, returns prioritized list with reasoning
6. Field worker executes distribution using the AI-ranked checklist
7. After distribution, field worker submits update form → FastAPI writes to local SQLite + queues sync job
8. When internet restored → sync worker pushes local changes to backend PostgreSQL DB
9. Gemma 4 online mode: function calling enables web search for enriched context (outbreak alerts, logistics data)

## 6. GEMMA 4 AI INTEGRATION

Gemma 4 E5B IT is the AI backbone of AidFlow Pro. The user has the model downloaded locally. It is served via **Ollama**, which exposes an OpenAI-compatible REST API at **http://localhost:11434/v1**. The application communicates with Gemma 4 using standard chat completions with carefully engineered system prompts.

## 6.1 Model Setup & Configuration

| Parameter | Value |
|---|---|
| Model Tag (Ollama) | gemma4:e5b-it (or gemma4:latest) |
| API Endpoint | http://localhost:11434/v1/chat/completions |
| Context Window | 128,000 tokens (E-series models) |
| Thinking Mode | Enabled for prioritization tasks (higher accuracy) |
| Max Response Tokens | 2,048 for prioritization; 512 for chat; 4,096 for document Q&A |
| Temperature | 0.2 for prioritization (deterministic); 0.7 for chat/assistant |
| Function Calling | Enabled — used for web search, database queries, weather alerts (online mode) |

## 6.2 System Prompt Design

Each use case has a tailored system prompt. Below are the key prompt templates Claude should implement in the application:

**Prioritization Prompt:**

```
You are AidFlow Pro's humanitarian aid prioritization AI. You receive structured JSON data about impacted families from a PostgreSQL database. Your job is to rank families by urgency for aid distribution. Score each family 0-100. Factors to consider: number of children under 5 (+20 pts each), pregnant/nursing women (+15), elderly members 65+ (+10 each), medical conditions (+25 for critical, +10 for chronic), days since last aid received (×2 pts per day), displacement status (+15 if recently displaced), family size weight (÷ by adults available). Return a JSON array sorted by score descending. Include a 1-sentence reason per family. Respond in {language}. If offline and context is limited, work with available data.
```

**RAG Assistant Prompt:**

```
You are a humanitarian field assistant for AidFlow Pro. You have access to organizational knowledge documents (uploaded PDFs). Use retrieved document excerpts to answer questions accurately. Always cite the document name and section. If the answer is not in the documents, say so clearly. Be concise and practical. Respond in {language}.
```

**Family Update Prompt:**

```
A humanitarian field worker has updated family #{family_id}'s status after an aid distribution. Previous priority score: {old_score}. Updates: {changes}. Recalculate the priority score using the same rubric. Explain what changed and why. Return JSON: {new_score, delta, reason}. Respond in {language}.
```

**Online-Enhanced Prompt:**

```
You are AidFlow Pro with internet access. In addition to local family data, you can call the web_search function to look up: disease outbreak alerts, weather events, conflict status, logistics updates. Use web search to enhance your prioritization when relevant. Always note which information came from web vs. local database. Respond in {language}.
```

## 6.3 RAG (Retrieval-Augmented Generation) Pipeline

- PDF Upload → PyMuPDF extracts text → chunked at 500 tokens with 50-token overlap
- Chunks embedded using local embedding model (nomic-embed-text via Ollama)
- Embeddings stored in ChromaDB (persistent local vector database)
- On query: user question → embed query → top-5 cosine-similar chunks retrieved
- Retrieved chunks injected into Gemma 4 prompt context alongside family data
- Gemma 4 response cites source document name and page range

## 7. OFFLINE-FIRST STRATEGY

AidFlow Pro is designed with a **Local-First Data Architecture**. The application must work in environments where internet may be unavailable for days or weeks. Every critical operation has an offline path.

| Feature | Offline Mechanism | Sync Strategy |
|---|---|---|
| Family Data Access | Full DB snapshot cached in IndexedDB (SQLite via sql.js) | Pull on sync on reconnect |
| AI Prioritization | Gemma 4 E5B IT runs on Ollama (localhost) — no internet needed | N/A (fully local) |
| PDF Knowledge Base | ChromaDB + vector embeddings stored on disk | N/A (fully local) |
| Distribution Updates | Write to local SQLite → queue for sync | Conflict-free merge (CRDT-based) |
| Map (Starlink/OSM) | Map tiles precached with Service Worker (Workbox) | Auto-refresh when online |
| Bitchat Messaging | Bluetooth LE mesh — no internet required | Nostr relay sync when online |
| Emotional Content Library | Media files cached in Cache API (Service Worker) | Download on WiFi, serve offline |
| Aid Usage Guides | PDF files cached via Service Worker | Download on WiFi, serve offline |
| App UI & Assets | PWA — full app shell cached on install | Background update check |

### Connectivity Status Indicator

A persistent banner at the top of the app shows the current connectivity mode: ■ **Online — Full AI** (internet + Gemma 4 with web tools), ■ **Local Mode** (no internet, Gemma 4 offline-only), or ■ **Disconnected** (Ollama not running — shows simplified rule-based fallback). The UI adapts — certain features (web search enrichment) are greyed out in offline mode.

## 8. BITCHAT COMMUNICATION INTEGRATION

Bitchat (github.com/permissionlesstech/bitchat) is a decentralized peer-to-peer messaging protocol that uses Bluetooth mesh networking — no internet, no servers, no phone numbers required. AidFlow Pro integrates a web-based Bitchat client to give field teams a secure communication channel in complete network outages.

### Integration Architecture

| Component | Implementation |
|---|---|
| Web Bluetooth API | Chrome/Edge on Android/Desktop — accesses BLE hardware from the browser |
| Message Protocol | Binary packet format per Bitchat spec — compact for BLE constraints |
| Encryption (Offline) | Noise Protocol — end-to-end encryption with forward secrecy |
| Encryption (Online) | NIP-17 gift-wrapped messages via Nostr relays |
| Fallback Transport | Nostr protocol over 290+ global relays (when internet available) |
| Multi-hop Relay | Automatic — messages route through nearby AidFlow Pro devices (max 7 hops) |
| Message Queuing | Smart queue — held locally until Bluetooth peer or Nostr relay available |

| | |
|---|---|
| Channels | Operational zones as named channels (e.g., #sector-b-north, #medical-team) |
| Family Notification | Field worker shares update codes with families via QR for channel access |

■ Browser Limitation Note: Web Bluetooth API is available in Chrome/Edge on Windows, macOS, Android, and Linux (with experimental flags). It is NOT available in Firefox or Safari. For full Bitchat support, field workers should use Chrome-based browsers. For iOS field workers, a companion native app wrapper (PWABuilder or Capacitor) is recommended.

## 9. STARLINK COVERAGE MAP MODULE

The map module provides humanitarian organizations with a visual planning tool to identify where Starlink equipment sellers, distributors, and coverage areas are located relative to their operation zone. This helps teams plan connectivity before deploying to the field.

- Base Map: OpenStreetMap tiles rendered with Leaflet.js — fully cached offline via Service Worker
- Coverage Overlays: GeoJSON polygons of approximate Starlink coverage zones (preloaded dataset, color-coded by signal strength)
- Provider Pins: Curated database of Starlink hardware sellers and installation providers with address, phone, hours
- User Location: Browser Geolocation API — 'Nearest providers to me' feature
- Custom Pins: Organizations can add their own Starlink installation points with notes (stored locally)
- Routing: 'Get Directions' links out to Google Maps / OpenStreetMap for provider navigation
- Offline Search: Filter providers by region/country from pre-indexed dataset (no internet search needed)
- Data Source: Community-curated Starlink reseller dataset + Starlink official coverage data (preloaded at build time)

## 10. MULTILINGUAL SUPPORT

| Language | Code | RTL? | UI Strings | AI Responses | Notes |
|---|---|---|---|---|---|
| English | en | No | ■ Full | ■ Full | Default / primary language |
| Arabic | ar | Yes ■ | ■ Full | ■ Full | RTL layout, critical for MENA crisis zones |
| French | fr | No | ■ Full | ■ Full | Central/West Africa operations |
| Spanish | es | No | ■ Full | ■ Full | Latin America / Caribbean operations |

Language switching is available in the top navigation bar at all times. The i18n system uses **react-i18next** with JSON translation files. Gemma 4 is instructed via system prompt to detect and respond in the active UI language. Arabic interface activates CSS **dir='rtl'** automatically.

## 11. DATABASE SCHEMA (ASSUMED BACKEND)

The backend PostgreSQL database is assumed to already exist. AidFlow Pro reads from it via a REST API and caches a local SQLite snapshot. Below are the key tables AidFlow Pro expects to consume:

### Table: families

| Column | Type | Description |
|---|---|---|
| family_id | UUID / INT PK | Unique family identifier |
| head_name | VARCHAR | Name of household head |
| member_count | INT | Total family members |
| children_under_5 | INT | Count of children under 5 years old |
| elderly_count | INT | Count of members 65+ years |
| has_pregnant_member | BOOLEAN | Presence of pregnant/nursing women |
| medical_conditions | JSONB / TEXT | JSON array of medical conditions |
| displacement_status | ENUM | 'resident' \| 'recently_displaced' \| 'refugee' |
| income_level | ENUM | 'none' \| 'minimal' \| 'moderate' |
| location_sector | VARCHAR | Operational sector/zone (e.g., 'Sector-B-North') |
| coordinates | POINT / VARCHAR | GPS coordinates for map display |
| last_updated | TIMESTAMP | Last record update timestamp |
| notes | TEXT | Free-form notes from field workers |

### Table: aid_distributions

| Column | Type | Description |
|---|---|---|
| distribution_id | UUID PK | Unique distribution event ID |

| | | |
|---|---|---|
| family_id | UUID FK | References families.family_id |
| session_id | UUID | Groups distributions in one field session |
| items_distributed | JSONB | JSON: {item_name, quantity, category} |
| distributed_by | VARCHAR | Field worker user ID |
| distributed_at | TIMESTAMP | Date and time of distribution |
| ai_priority_score | FLOAT | Gemma 4 priority score at time of distribution |
| ai_reasoning | TEXT | Gemma 4 explanation for the score |
| post_update_notes | TEXT | Notes added after distribution |
| new_needs_flagged | BOOLEAN | Whether field worker flagged new urgent needs |

### Additional Tables (Referenced)

| Table | Purpose |
|---|---|
| users | Organization staff accounts with roles and permissions |
| documents | Metadata for uploaded PDFs (knowledge base, usage guides, emotional content) |
| aid_inventory | Current stock of available aid items per distribution point |
| sessions | Distribution session records (start/end time, sector, supervisor) |
| sync_log | Offline/online sync event log for audit trail |

## 12. UI/UX DESIGN SPECIFICATIONS

### Design Principles

- Field-First: Large touch targets (min 44×44px), high contrast, works on dusty screens and gloves
- Information Hierarchy: Priority scores dominate — color-coded (■ Critical 80-100, ■ High 60-79, ■ Medium 40-59, ■ Normal <40)
- Offline Clarity: Always-visible connectivity status banner — users never confused about current mode
- Accessibility: WCAG 2.1 AA compliance, screen reader support, 4.5:1 contrast ratios minimum
- Minimal Cognitive Load: Workflows broken into 3-step wizards — no complex multi-field forms

### Screen Inventory

| Screen | Route | Description |
|---|---|---|
| Login / Auth | /login | PIN or password login; offline session token stored locally |
| Dashboard | /dashboard | KPI cards: families prioritized today, items distributed, AI status, sync status |
| Family List | /families | AI-sorted family list with priority badges, search, sector filter |
| Family Detail | /families/:id | Full profile, aid history, AI score breakdown, Gemma 4 chat about this family |
| Distribution Session | /distribute | Step 1: Select sector → Step 2: AI-ranked list → Step 3: Mark delivery → S |
| AI Assistant | /assistant | Full-screen Gemma 4 chat — ask questions about families, guides, protocols |
| Knowledge Base | /docs | PDF library viewer, upload, categories, search — full RAG access |
| Emotional Content | /kids | Media library for children — filtered by age/language, offline playback |
| Aid Guides | /guides | Usage guide library for distributed aid items — Gemma 4 Q&A enabled |
| Starlink Map | /map | Leaflet map with Starlink provider pins and coverage overlays |
| Bitchat | /chat | Offline mesh communication — zone channels and direct messages |
| Settings | /settings | Language, theme, AI model status, sync schedule, user management (adm |
| Reports | /reports | Distribution history, export CSV/PDF, Gemma 4 summary reports |

### Color System

| Token | Hex | Usage |
|---|---|---|
| Priority Critical | #ef4444 (red-500) | Family priority score 80-100 — immediate distribution needed |
| Priority High | #f97316 (orange-500) | Family priority score 60-79 |
| Priority Medium | #eab308 (yellow-500) | Family priority score 40-59 |
| Priority Normal | #22c55e (green-500) | Family priority score below 40 |
| Primary Brand | #0ea5e9 (sky-500) | Buttons, links, active states |
| AI Accent | #8b5cf6 (violet-500) | AI-generated content, Gemma 4 responses |

| | | |
|---|---|---|
| Offline Warning | #f59e0b (amber-500) | Connectivity status banner (local mode) |
| Background | #0f172a (slate-900) | Dark mode (default for field use — OLED battery saving) |
| Surface | #1e293b (slate-800) | Cards, panels |
| Text Primary | #f1f5f9 (slate-100) | Main body text (dark mode) |

## 13. TECHNOLOGY STACK

| Category | Technology | Version / Notes |
|---|---|---|
| Frontend Framework | React | v18 + TypeScript |
| Build Tool | Vite | v5 — fast HMR, PWA plugin (vite-plugin-pwa) |
| Styling | TailwindCSS | v3 — utility-first, dark mode, RTL support |
| State Management | Zustand | Lightweight, offline-friendly store |
| Routing | React Router | v6 — SPA routing |
| PWA / Service Worker | Workbox (via Vite) | Precaching, background sync, offline fallback |
| Offline Database (client) | Dexie.js (IndexedDB) | Reactive queries, sync queue, large data support |
| Maps | Leaflet.js + react-leaflet | v1.9 — tile caching, GeoJSON overlays, offline tiles |
| i18n | react-i18next | v14 — EN/AR/FR/ES, RTL toggle |
| Communication (Bitchat) | Web Bluetooth API | Native browser API — Chrome/Edge required |
| Charts / Dashboards | Recharts | v2 — priority distribution charts, aid history graphs |
| Backend Framework | FastAPI (Python) | v0.111 — async, OpenAPI docs auto-generated |
| Local AI Server | Ollama | Latest — serves Gemma 4 E5B IT at localhost:11434 |
| AI Model | Gemma 4 E5B IT | User's locally downloaded model — Apache 2.0 license |
| LLM Integration | LangChain (Python) | v0.2 — chains, RAG pipeline, function calling |
| Vector Database | ChromaDB | Persistent local store — PDF embeddings |
| Embedding Model | nomic-embed-text (Ollama) | Local embedding — no cloud required |
| Relational DB (local) | SQLite via SQLAlchemy | Local sync cache of backend data |
| PDF Processing | PyMuPDF (fitz) | Text extraction from uploaded PDFs |
| Auth | JWT + bcrypt | Stateless tokens, offline-compatible |
| Containerization | Docker + docker-compose | One-command deployment for organizations |
| Backend DB (assumed) | PostgreSQL | Existing — not built, only consumed via API |
| Sync Protocol | Custom REST + WebSocket | Offline queue → online sync on reconnect |
| File Storage | Local filesystem / S3-compatible | PDF, media uploads — local or MinIO |
| Deployment Target | Local server / laptop / Raspberry Pi | Organization's own hardware — no cloud required |

## 14. SECURITY & PRIVACY

- **Local-First Data:** All family data is stored on the organization's own hardware. Nothing is sent to external AI APIs — Gemma 4 runs entirely locally. This protects vulnerable people's sensitive information.
- **JWT Authentication:** Role-based access control (RBAC) with JWT tokens. Tokens stored securely in httpOnly cookies. Offline session tokens valid for 7 days — refreshed on reconnect.

- **Data Encryption at Rest:** Local SQLite database encrypted with SQLCipher. IndexedDB data protected by browser origin isolation.
- **Bitchat Encryption:** All Bitchat messages encrypted with Noise Protocol (offline Bluetooth mesh) — end-to-end, forward secrecy. Online messages use NIP-17 gift-wrapping.
- **PDF Document Access:** Uploaded documents accessible only to authenticated users with appropriate role. No document content is ever sent to external services.
- **Audit Trail:** All distribution events, AI decisions, and data updates logged with user ID, timestamp, and device fingerprint for accountability.
- **GDPR / Humanitarian Privacy:** Follows ICRC digital data protection guidelines. Family data minimum-collected, purpose-limited, with right to deletion by data managers.

## 15. DEVELOPMENT ROADMAP & MILESTONES

The development is planned as 4 phases. The Hackathon MVP (Phases 1-2) is the primary target, with Phase 3-4 as post-hackathon enhancements.

### Phase 1 — Foundation & Core AI (Week 1)

- Project scaffolding: Vite + React + TypeScript + TailwindCSS + PWA plugin
- FastAPI backend setup with SQLite local cache and REST API endpoints
- Ollama integration — Gemma 4 E5B IT chat completions working
- Authentication system (JWT, user roles, offline tokens)
- Family list view with mock data — prioritization API call → display results
- Basic AI assistant chat interface connected to Gemma 4
- Offline mode detection + connectivity status banner
- Docker compose setup for one-command deployment

### Phase 2 — Full Feature Set (Week 2)

- RAG pipeline: PyMuPDF + ChromaDB + nomic-embed-text — PDF upload and Q&A; working
- Distribution execution workflow (3-step wizard with family update)
- Priority score recalculation after distribution update
- Starlink map (Leaflet.js + offline tile caching + provider pins GeoJSON)
- Bitchat web client integration (Web Bluetooth API bridge)
- Multilingual support: i18n for EN/AR/FR/ES + RTL Arabic layout
- Emotional support content library (upload, categorize, offline serve)
- Aid usage guide library with Gemma 4 Q&A;
- Service Worker full offline caching (Workbox)
- Background sync queue for offline writes

### Phase 3 — Polish & Hackathon Submission (Days 1-2 of Week 3)

- Dashboard KPI charts and distribution history reports
- Full RBAC enforcement across all routes
- Mobile responsiveness testing and touch target optimization
- Gemma 4 online-enhanced mode (function calling + web search)
- Accessibility audit (WCAG 2.1 AA compliance checks)
- Public GitHub repository setup + README + code comments
- Technical write-up document (this plan + implementation notes)
- Demo video recording (3-5 minutes showing all key features)
- Kaggle submission form completion before May 18, 2026 deadline

### Phase 4 — Post-Hackathon (Future)

- Native iOS/Android wrapper via Capacitor (full Bluetooth support on iOS)
- Gemma 4 fine-tuning on humanitarian domain data using Unsloth

- Real Starlink API integration for live coverage data
- Multi-organization support (tenant isolation)
- Advanced analytics: aid gap analysis, population health trends
- UNHCR / ICRC data format compatibility (Kobo, ODK, ActivityInfo)

## Critical Path Dependencies

- Ollama + Gemma 4 E5B IT must be running locally before any AI features can be tested
- Web Bluetooth API requires Chrome/Edge — test Bitchat integration on Android tablet first
- Service Worker caching must be tested in production build mode (Vite --mode production)
- ChromaDB persistence directory must be configured before PDF uploads are tested
- PostgreSQL assumed backend must provide a REST API spec — mock with JSON Server for hackathon demo

## 16. HACKATHON ALIGNMENT & IMPACT METRICS

### Gemma 4 Good Hackathon — Competition Details

| Criterion | AidFlow Pro Response |
|---|---|
| Competition | Gemma 4 Good Hackathon — Kaggle × Google DeepMind |
| Prize Pool | $200,000 total — General, Impact, Technical, and Unsloth (fine-tuning) categories |
| Submission Deadline | May 18, 2026 |
| Track | Global Resilience — explicitly targets humanitarian and disaster response use cases |
| Gemma 4 Usage | Core AI engine — all prioritization, RAG, and assistant features use Gemma 4 E5B IT loc |
| Working Demo | Full PWA demo with mock family database, live AI prioritization, offline mode toggle |
| Public Code Repo | GitHub repository with setup instructions, Docker compose, API docs |
| Technical Write-Up | This document + implementation notes in repository README |
| Demo Video | 3-5 minute walkthrough showing all 8 key features, offline mode, and Gemma 4 reasoning |

### Judging Criteria Alignment

| Criterion (Weight) | How AidFlow Pro Scores |
|---|---|
| Real-World Impact (40%) | Addresses a life-or-death problem affecting 300M+ crisis-affected people globally. Demon |
| Technical Execution (35%) | Gemma 4 local inference via Ollama, RAG with ChromaDB, offline-first PWA, Bluetooth m |
| Communication / Use Case Clarity (25%) | Clear narrative: humanitarian workers in field zones with destroyed infrastructure need AI |

### Measurable Impact Metrics (for Demo)

- Priority Score Accuracy: Demo shows Gemma 4 correctly ranks most-vulnerable family first given identical inputs
- Distribution Speed: AI-ranked list allows field worker to service 50 families 3x faster than manual sorting
- Offline Resilience: App demonstrated working with no internet for 100% of core features
- Knowledge Retrieval: Gemma 4 answers a medical protocol question citing the correct uploaded PDF
- Multilingual: Full Arabic RTL interface shown alongside English mode
- Post-Distribution Update: Show priority score changing in real-time after family status update

## 17. DEPLOYMENT GUIDE FOR CLAUDE AI BUILDER

This section provides explicit instructions for Claude (the AI assistant building this project) on how to structure, prioritize, and implement AidFlow Pro.

### 17.1 Repository Structure

| Path | Contents |
|---|---|

| | |
|---|---|
| aidflow-pro/ | Root monorepo directory |
| ■■■ frontend/ | Vite + React PWA application |
| ■ ■■■ src/ | React components, hooks, stores, i18n |
| ■ ■ ■■■ pages/ | One folder per screen (Dashboard, Families, Distribute, etc.) |
| ■ ■ ■■■ components/ | Shared UI components (PriorityBadge, FamilyCard, AIChat, etc.) |
| ■ ■ ■■■ stores/ | Zustand stores (families, auth, sync, connectivity) |
| ■ ■ ■■■ services/ | API client, Ollama client, Dexie DB, i18n config |
| ■ ■ ■■■ locales/ | en.json, ar.json, fr.json, es.json translation files |
| ■ ■■■ public/ | Static assets, map tiles (offline), service worker |
| ■ ■■■ vite.config.ts | Vite + PWA plugin configuration |
| ■■■ backend/ | FastAPI Python application |
| ■ ■■■ app/ | FastAPI routes, models, services |
| ■ ■ ■■■ routers/ | families.py, distributions.py, ai.py, documents.py, auth.py |
| ■ ■ ■■■ services/ | ollama_client.py, rag_service.py, sync_service.py |
| ■ ■ ■■■ models/ | SQLAlchemy models for local SQLite cache |
| ■ ■ ■■■ core/ | config.py, security.py, database.py |
| ■ ■■■ chroma_db/ | ChromaDB persistent vector store (gitignored) |
| ■ ■■■ requirements.txt | Python dependencies |
| ■■■ docker-compose.yml | Orchestrates frontend, backend, Ollama, ChromaDB |
| ■■■ .env.example | Environment variable template |
| ■■■ README.md | Setup guide, architecture overview, hackathon notes |

### 17.2 Build Order (For Claude)

**Step 01:** Start with backend: FastAPI app skeleton + SQLite models + `/api/families` endpoint returning mock JSON

**Step 02:** Add Ollama client: `ollama_client.py` with `prioritize_families()` and `chat()` functions using openai-python pointed at localhost:11434

**Step 03:** Build frontend auth: Login page, JWT store in Zustand, offline token in localStorage

**Step 04:** Build Family List page: fetch from `/api/families`, call AI prioritize endpoint, render sorted list with PriorityBadge

**Step 05:** Build AI Assistant page: chat interface sending messages to `/api/ai/chat` which calls Gemma 4

**Step 06:** Add RAG pipeline: `rag_service.py` with ChromaDB + nomic-embed-text embedding + document Q&A; endpoint

**Step 07:** Build Document Upload page: PDF upload → process → embed → confirm stored in ChromaDB

**Step 08:** Add PWA config: vite-plugin-pwa + Workbox precaching + Service Worker for offline tiles

**Step 09:** Add Dexie.js: offline IndexedDB store for families, sync queue for pending updates

**Step 10:** Build Distribution Workflow: 3-step wizard → update form → POST to `/api/distributions` → trigger score recalculation

**Step 11:** Build Starlink Map: Leaflet.js + GeoJSON provider data + OSM tiles + offline caching

**Step 12:** Add Bitchat: Web Bluetooth API integration with Bitchat BLE protocol bridge

**Step 13:** Add i18n: react-i18next setup + all 4 language JSON files + RTL Arabic toggle

**Step 14:** Build Kids Content library + Aid Guides library (same upload pattern as documents)

**Step 15:** Finalize Docker compose + environment variables + README

**Step 16:** Build Reports/Dashboard with Recharts charts

**Step 17:** Final QA: test offline mode (disable network in DevTools), test AI responses, test RTL Arabic

### 17.3 Key Environment Variables

| Variable | Value | Notes |
|---|---|---|
| OLLAMA_BASE_URL | http://localhost:11434 | Gemma 4 Ollama server |
| OLLAMA_MODEL | gemma4:e5b-it | User's downloaded model tag |
| EMBEDDING_MODEL | nomic-embed-text | Local embedding for RAG |
| CHROMA_PERSIST_DIR | ./chroma_db | ChromaDB vector store path |
| DATABASE_URL | sqlite:///./aidflow.db | Local SQLite cache |
| BACKEND_DB_URL | postgresql://... | Assumed existing PostgreSQL |
| JWT_SECRET_KEY | (random 32-char string) | Auth token signing key |
| UPLOAD_DIR | ./uploads | PDF and media file storage |
| VITE_API_BASE_URL | http://localhost:8000 | Frontend → Backend API URL |

## 18. APPENDIX

### A. Key External References

| Resource | URL / Reference |
|---|---|
| Gemma 4 Official Docs | ai.google.dev/gemma/docs/core |
| Gemma 4 Good Hackathon | kaggle.com/competitions/gemma4good (deadline May 18, 2026) |
| Ollama Gemma Integration | ai.google.dev/gemma/docs/integrations/ollama |
| Bitchat Protocol | github.com/permissionlesstech/bitchat |
| Bitchat Website | bitchat.free |
| Ollama API Docs | github.com/ollama/ollama/blob/main/docs/api.md |
| LangChain RAG Guide | python.langchain.com/docs/use_cases/question_answering |
| ChromaDB Docs | docs.trychroma.com |
| Vite PWA Plugin | vite-pwa-org.netlify.app |
| Workbox Offline | developer.chrome.com/docs/workbox |
| Web Bluetooth API | developer.chrome.com/docs/capabilities/bluetooth |
| react-i18next | react.i18next.com |
| Leaflet.js | leafletjs.com |
| Dexie.js (IndexedDB) | dexie.org |
| OpenStreetMap Tiles | tile.openstreetmap.org |
| ICRC Digital Data Protection | icrc.org/en/doc/assets/files/other/icrc-002-4030.pdf |

### B. Gemma 4 E5B IT — Local Capability Summary

| Property | Value |
|---|---|
| Model Name | Gemma 4 E5B IT (Instruction-Tuned) |
| Parameter Count (Effective) | ~5 billion activated per token |
| Architecture | Mixture-of-Experts (MoE) edge-optimized variant |
| Context Window | 128,000 tokens |
| Multimodal Support | Text + Image + Audio (native) |
| Function Calling | Yes — natively supported for agentic workflows |
| Thinking Mode | Configurable — enhanced reasoning for complex tasks |
| Offline Capable | Yes — runs fully on local hardware via Ollama |
| License | Apache 2.0 — free for commercial and hackathon use |
| Hardware Requirement | 8GB+ VRAM GPU or 16GB+ RAM CPU (user has this confirmed) |

| | |
|---|---|
| Ollama Command | ollama run gemma4:e5b-it |

### C. Prioritization Scoring Algorithm (Reference)

The following scoring rubric guides Gemma 4's prioritization decisions. These weights are injected into the system prompt and can be adjusted by organization admins.

| Factor | Points Added | Rationale |
|---|---|---|
| Each child under 5 years old | +20 pts each | Highest mortality risk in food crises |
| Pregnant or nursing women | +15 pts per person | Critical nutrition window — double burden |
| Each elderly member (65+) | +10 pts each | Higher vulnerability to malnutrition complications |
| Critical medical condition | +25 pts | Requires immediate intervention |
| Chronic medical condition | +10 pts | Ongoing vulnerability |
| Days since last aid received | +2 pts per day | Deprivation accumulates linearly |
| Recently displaced (< 30 days) | +15 pts | No established support network |
| Refugee status (active) | +10 pts | Structural disadvantage |
| No income at all | +15 pts | Cannot self-supplement |
| Minimal income | +5 pts | Limited self-help capacity |
| New need flagged by field worker | +20 pts (one-time) | Urgent alert from trusted source |
| Received aid < 3 days ago | -30 pts | Recently served — deprioritize |
| Large family (> 8 members) | ÷ 0.9 final score | More distribution points per visit |

### D. Sample Gemma 4 Prioritization Output (Expected Format)


```
[ { "family_id": "F-0042", "head_name": "Ahmed Al-Rashid", "priority_score": 94, "priority_level":
"CRITICAL", "reason": "3 children under 5, 1 pregnant mother, no aid received in 18 days, recently
displaced.", "recommended_items": ["infant formula", "high-protein rations", "prenatal
supplements"], "sector": "Sector-B-North" }, { "family_id": "F-0089", "priority_score": 67,
"priority_level": "HIGH", "reason": "2 elderly members (72, 68), 1 critical medical condition
(diabetes), 9 days without aid.", "recommended_items": ["diabetic-safe rations", "medical kit",
"water purification"], "sector": "Sector-A-South" } ]
```


---

*AidFlow Pro — Built for the Gemma 4 Good Hackathon | Global Resilience Track Powered by Gemma 4 E5B IT (Google DeepMind) running locally via Ollama Offline-first. Human-centered. Life-saving.*